

分布式多人实时协同绘画工具

概要设计文档

(High-Level Design Document)

设计题目：分布式多人实时协同绘画工具

专业方向：软件工程

2026 年 5 月

目录

第一章 总体架构设计	1
1.1 架构演进思想	1
1.2 系统分层物理拓扑	2
第二章 数据结构与通信协议设计	4
2.1 画布矢量操作日志模型 (Event Sourced Model)	4
2.1.1 MongoDB 数据模型	4
2.2 跨节点实时通信协议设计 (Payload Protocol)	5
2.3 Redis 房间状态缓存层	6
第三章 分布式核心机制与时序设计	7
3.1 跨节点笔触广播时序链路	7
3.2 绘画冲突与一致性保障方案 (CRDT / 时序控制)	7
3.2.1 服务器端操作序列化	8
3.2.2 图层叠加策略	8
3.2.3 CRDT 视角的建模	8
3.3 消息节流与网络优化 (Throttling & Batching)	9
3.3.1 前端节流 (Throttling)	9
第四章 接口设计规范	11
4.1 HTTP RESTful 接口设计	11
4.2 WebSocket 事件交互协议	11
第五章 故障演练与压测预案	13
5.1 核心监控大盘指标 (Observability)	13
5.2 分布式压测场景设计	13
5.2.1 场景一：200 人同房高频作画压测	13

目录	II
5.2.2 场景二：跨节点断线重连压测	14
5.2.3 场景三：Kafka 故障降级压测	14
第六章 系统部署与运维	16
6.1 容器化部署架构	16

第一章 总体架构设计

1.1 架构演进思想

传统绘画应用的核心误区在于将画布状态存储为**静态位图 (Bitmap)**——即 PNG、JPEG 等二进制图像文件。这种存储模式在单用户场景下是自然的，但一旦引入多人协同，位图的并发合并问题就变得极其棘手：

当两名用户同时在画布左上角和右下角作画时，若后端以 Last-Write-Wins 策略直接覆盖位图，则最后写入的用户会覆盖前一位用户的笔触；若采用锁机制，则任意时刻只能有一人作画，并发体验降为零。

本项目采纳了**事件溯源 (Event Sourcing)**的设计思想：系统**不存储”死图片”，而是存储”活的操作流”**。每一笔绘画不再是像素的堆叠，而是一组具有语义的操作指令（“在坐标 (x_1, y_1) 到 (x_n, y_n) 之间，以颜色 C 、线宽 W 绘制贝塞尔曲线”）。画布的当前状态并非存储出来的，而是通过重放 (Replay) 历史操作流从零计算出来的。

这一架构选型带来了三个核心优势：

1. **天然支持并发合并**：所有操作指令天然顺序无关的——无论两条 Stroke 的绘制时间多么接近，只要按服务器分配的全局 Sequence_ID 排序重放，所有客户端都将收敛至同一状态。
2. **存储成本极低**：一条 Stroke 的存储大小 (JSON/Protobuf) 远小于同等效果的位图图像存储。按每条 Stroke 平均 200 字节计算，100 万条操作仅占用约 200MB 存储，而同等精度的位图可能需要数十 GB。
3. **完美支持回放与审计**：操作流天然支持任意倍速回放、时间旅行 (Time Travel) 和操作审计，无需额外的录制基础设施。

在读写分离层面，系统采用 **Redis Pub/Sub + MongoDB** 的双层架构：Redis 承担实时消息总线的角色，保证跨节点广播的亚毫秒级延迟；MongoDB 承担操作日志的持久化存储，保证数据的最终安全。Kafka 作为两者之间的异步缓冲层，实现了写入路径与持久化路径的解耦，避免了高频写入直接击穿 MongoDB。

1.2 系统分层物理拓扑

图 1-1 描述了系统的整体物理架构拓扑，包含 Nginx 负载均衡层、WebSocket 网关集群、Redis 消息总线、Kafka 持久化缓冲层与 MongoDB 存储层。

拓扑说明：

- **客户端层：**所有用户浏览器均通过标准 WebSocket 协议接入，无需安装额外插件。Canvas 渲染引擎负责矢量路径的本地绘制。
- **Nginx 负载均衡层：**部署至少两个 Nginx 实例 (Active-Standby 或 Active-Active 模式)，通过 `tcp_upstream` 模块实现 WebSocket 连接的四层负载均衡。Nginx 负责 TLS 终止 (TLS Termination)，并将解密后的明文流量转发至后端网关集群。
- **WebSocket 网关集群：**每个网关节点 (Go/Java 实现) 维护与其连接的所有客户端的会话状态。节点之间通过 Redis Pub/Sub 进行房间级别的消息广播，无需点对点直连。
- **消息总线层：**Redis Cluster 负责房间广播的实时消息传递；Kafka 负责操作日志的异步持久化。两者各司其职：Redis 管“快”，Kafka 管“稳”。
- **存储层：**MongoDB 存储完整的操作事件流 (Event Store) 与定期快照；Redis 同时承担 `Sequence_ID` 的原子生成 (计数器)、房间成员列表缓存以及近期操作流的临时缓存。

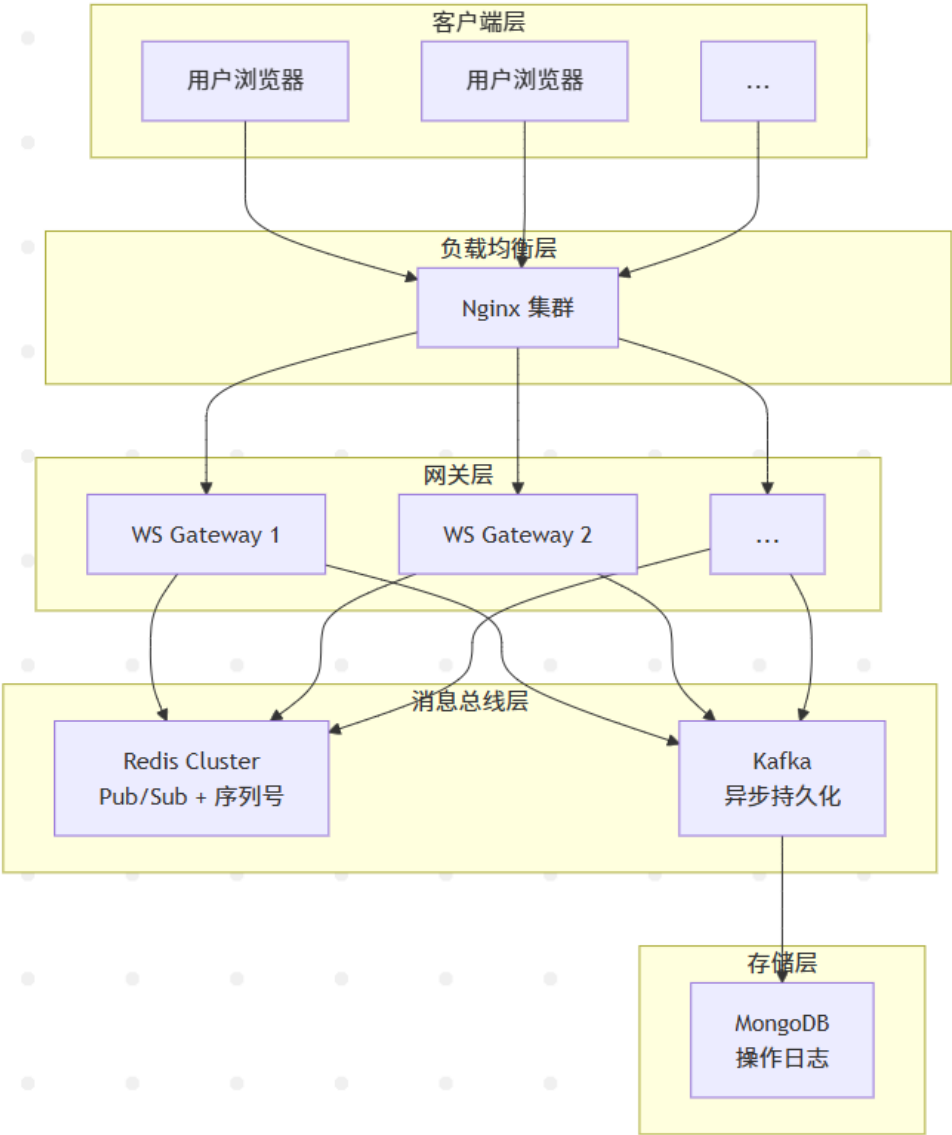


图 1.1: 分布式协同绘画工具系统架构拓扑图

第二章 数据结构与通信协议设计

2.1 画布矢量操作日志模型 (Event Sourced Model)

本系统不存储位图像素，所有画布数据均以不可变矢量操作事件的序列形式存储于 MongoDB。每条记录代表用户在画布上执行的一次完整绘画行为。

2.1.1 MongoDB 数据模型

MongoDB 中 operations 集合的文档结构设计如下：

表 2.1: operations 集合文档结构

字段名	数据类型	索引	说明
_id	ObjectId	主键	MongoDB 自动生成的文档唯一标识符
sequence_id	Long	唯一索引	服务器分配的全局单调递增序列号，64 位无符号整数，由 Redis INCR 原子生成
room_id	String	房间索引	所属房间 UUID v4 标识符
user_id	String	用户索引	操作发起者用户标识符
operation_type	String	类型索引	操作类型枚举：stroke_start、stroke_move、stroke_end、eraser、clear_canvas
tool_type	String	—	工具类型：brush、pencil、eraser、spray、shape
color	String	—	十六进制颜色代码，如 #FF5733
width	Double	—	线条宽度（像素），精度至小数点后 2 位
points	Array	—	贝塞尔曲线控制点数组，每点为 {x, y, pressure}
stroke_id	String	房间索引	笔触全局唯一标识符（UUID v4）
timestamp	Date	时间索引	客户端本地时间戳（毫秒）
server_timestamp	Date	时间索引	服务器接收时的 UTC 时间戳

其中 points 数组内的每个 Point 结构为 {x, y, pressure}，pressure 字段支持压感

笔力度值 (0.0~1.0)，普通鼠标固定为 1.0。

快照 (Snapshot) 集合结构：

表 2.2: snapshots 集合文档结构

字段名	数据类型	索引	说明
_id	ObjectId	主键	快照唯一标识符
room_id	String	房间索引	所属房间 UUID v4 标识符
sequence_id	Long	唯一索引	此快照截止的 sequence_id
canvas_width	Integer	—	画布宽度（像素）
canvas_height	Integer	—	画布高度（像素）
svg_data	String	—	画布当前状态 SVG 矢量描述字符串
created_at	Date	时间索引	快照创建时间

2.2 跨节点实时通信协议设计 (Payload Protocol)

WebSocket 传输层与 Redis Pub/Sub 层的消息 Payload 采用统一设计。在带宽敏感场景下（如移动端），可切换为 Protocol Buffers (Protobuf) 二进制编码以压缩体积。以下以 JSON 格式说明协议结构：

WebSocket 传输层与 Redis Pub/Sub 层的消息 Payload 采用统一设计，带宽敏感场景下可切换为 Protocol Buffers 二进制编码。以下以 JSON 格式说明协议结构：

msg_type(String, 必选)为消息类型标识,枚举值包括 stroke_start、stroke_move、stroke_end、cursor_sync、user_joined、user_left、canvas_cleared、ping/pong、sync_request。
room_id (String, 必选) 为房间 UUID v4 标识符, Redis Pub/Sub 以此作为 Channel 命名依据 (格式 room:{room_id})。user_id (String, 必选) 为操作发起者用户标识符 (从 JWT 中提取)。sequence_id (Long, 条件必选) 仅在下行消息 stroke_update 中必选, 由网关统一分配后附加到操作日志记录中。points (Array, 条件必选) 为坐标增量点数组, 仅在 stroke_start/move/end 时存在, 每次最多携带 50 个点以防止单包过大, 格式为 {x, y, pressure}。color (String, 条件必选) 为笔触颜色, 仅在 stroke_start 时携带, 后续 move/end 消息复用。width (Double, 条件必选) 为笔触宽度, 范围 0.5~100.0 像素, 仅在 stroke_start 时携带。tool (String, 条件必选) 为工具类型, 仅在 stroke_start 时携带。stroke_id (String, 条件必选) 为笔触 UUID v4 标识符, stroke_start 时生成, 同笔触的 move/end 复用。cursor (Object, 条件必选) 仅在 cursor_sync 时存在, 格式为 {x, y}。timestamp (Long, 必选) 为客户端本地毫秒时间戳, 用于计算 RTT 与光标动画插值, 服务器不依赖此字段排序。local_op_id

(String, 可选) 为客户端本地操作唯一标识符, 用于幂等性校验与去重。

2.3 Redis 房间状态缓存层

为应对突发的海量新连接请求 (如直播场景下的观众涌入), 系统利用 Redis 实现**近期操作流缓存层**, 避免瞬间击穿 MongoDB:

- **缓存结构:** Redis ZSet, Key 命名格式为 `room:{room_id}:ops`, Score 为 `Sequence_ID`, Value 为 JSON 序列化后的操作消息。TTL 设置为 5 分钟 (300 秒)。
- **写入策略:** 每条新操作在写入 Kafka 的同时, 以 ZADD 命令追加至 Redis ZSet。ZSET 自动按 `Sequence_ID` 排序, 支持范围查询。
- **读取策略:** 新用户入房时, 首先查询 Redis ZSet 获取近期操作; 若 Redis 中数据不足 (`sequence_id` 差距过大), 再降级查询 MongoDB。
- **容量控制:** 每个房间的 ZSet 最多保留最近 5,000 条操作 (约覆盖最近 1-2 分钟的高频绘画), 超出部分通过 ZREMRANGEBYRANK 自动清理。

此外, Redis 还承担以下职责:

- **全局序列号生成器:** Redis 字符串键 `global:sequence`, 通过 INCR 命令原子递增, 返回值作为每条操作的 `Sequence_ID`。所有网关节点共享此计数器。
- **房间成员列表:** Redis Hash, `room:{room_id}:members`, Field 为 `user_id`, Value 为 JSON 序列化的用户状态 (角色、光标位置、最后活跃时间)。

第三章 分布式核心机制与时序设计

3.1 跨节点笔触广播时序链路

图 3-1 描述了一条笔触操作从用户 A（在 WebSocket 网关节点 1 上）发起，到达用户 B（在网关节点 2 上）并完成渲染的完整链路。

时序链路关键说明：

1. 用户 A 的绘画操作首先在本机捕获并立即渲染（乐观更新），避免等待网络往返带来的延迟感知。
2. WS Gateway Node-1 通过 Redis INCR 命令获取全局唯一的 Sequence_ID，此操作是原子性的保证了多节点间的序列号唯一性。
3. 操作通过 Redis Pub/Sub 发布至房间频道。Redis Cluster 的内部复制机制确保消息在多个副本节点间同步，即使某一 Redis 节点宕机，Pub/Sub 消息也不会丢失。
4. Kafka 的写入采用异步发送（Async Send），网关节点无需等待 Kafka ACK 返回即可继续处理下一条消息，确保了绘画操作的低延迟。
5. WS Gateway Node-2 订阅同一房间频道，收到消息后立即转发给连接在自己身上的用户 B，无需额外处理。
6. MongoDB 的写入由独立的 Kafka Consumer Group 执行，批量写入（Batch Insert）降低了数据库 I/O 频率。

3.2 绘画冲突与一致性保障方案（CRDT / 时序控制）

多人在极近的坐标同时作画时（如两人同时在画布中心绘制），会产生并发冲突。本节详细论述前端与后端的协同冲突处理策略。

3.2.1 服务器端操作序列化

系统的**核心一致性原则**是：所有客户端必须严格按照服务器分配的全局单调递增 Sequence_ID 执行操作，任何客户端不得自行决定操作顺序。

当多条操作几乎同时到达网关时（例如两个网关节点同时收到不同用户的操作），Redis INCR 命令保证了即使在高并发场景下，Sequence_ID 的分配也是严格有序的。例如：用户 A 的操作分配到 seq=10001，用户 B 的操作分配到 seq=10002，无论两人在物理空间上多么接近，时间上多么接近，seq 的严格单调性确保了所有客户端看到的操作顺序完全一致。

3.2.2 图层叠加策略

Canvas 2D 的默认渲染模型是**后绘制覆盖先绘制**（Painter's Algorithm）。这在协同场景下天然实现了“最终结果以最后到达服务器的操作优先”的语义，即一种简化的 LWW（Last Write Wins）策略。

具体而言，当用户 A 和用户 B 在同一区域同时作画时：

1. 两人的笔触分别分配了 seq=10001 和 seq=10002。
2. 所有客户端均按 seq 升序依次渲染两个笔触。
3. 最终画布上两人的笔触均完整保留，叠加显示——这正是协同绘画的期望行为。
4. 若需要更细粒度的冲突解决（如两人绘制了重叠的像素点），则在 Canvas 2D 的 alpha 混合模式下自然融合，无需额外处理。

3.2.3 CRDT 视角的建模

从 CRDT 的角度审视，Canvas 上的每一条 Stroke 可以建模为一个**增量的 LWW-Register**：Stroke 的内容（坐标点、颜色、宽度）即 Register 的值，当有新的 Stroke 加入时，通过 Union 操作合并到画布上。由于每条 Stroke 是独立的不可变对象（Immutable），而非对同一区域的可变修改，因此不存在传统意义上的 CRDT 冲突。

真正需要 CRDT 处理的是**图层属性**（Layer Properties）的并发修改，例如：用户 A 修改图层 1 的透明度，用户 B 同时修改图层 1 的颜色。此时采用 Field-Level CRDT——每个可修改属性（透明度、颜色、位置）各自维护一个 LWW-Register，以最后一次到达服务器的修改值为准。

3.3 消息节流与网络优化 (Throttling & Batching)

鼠标移动事件在标准浏览器环境中每约 16ms 触发一次（对应 60Hz 屏幕刷新率）。在未做任何优化的情况下，一名用户移动鼠标 1 秒将产生约 60 条坐标事件，每条消息平均携带数十个坐标点——这将在极短时间内耗尽网络带宽。

本节详述前端与后端的双层节流优化方案。

3.3.1 前端节流 (Throttling)

1. **固定间隔节流 (Fixed Interval)**: 放弃监听每一条 `mousemove` 事件，改为使用 `setInterval` 或 `requestAnimationFrame` 在固定时间间隔（推荐 30ms，即约 33 次/秒）内采样一次鼠标位置。
2. **坐标点累积批处理 (Batching)**: 在一个发送间隔内，将所有捕获的鼠标坐标点暂存于本地数组（Accumulator）中；到达发送时刻时，将数组整体打包为一条 `stroke_move` 消息发送。
3. **贝塞尔曲线平滑**: 在打包前，对累积的坐标点应用 Catmull-Rom Spline 或二次贝塞尔曲线插值，减少点密度的同时保持曲线的视觉平滑度。
4. **动态节流速率**: 根据当前网络状态动态调整节流间隔——检测到 RTT 上升（连续 3 个心跳超过 200ms）时，自动将发送间隔从 30ms 放宽至 50ms 或 100ms，降低发送频率以缓解网络拥塞。

前端节流的核心实现思路是：在固定时间间隔（推荐 30ms）内，将捕获的鼠标坐标点暂存于数组 Accumulator 中；到达发送时刻时，将数组整体打包为一条 `stroke_move` 消息发送。同时在打包前对坐标点应用贝塞尔曲线插值以保持视觉平滑度，并根据网络 RTT 动态调整节流间隔——当连续 3 个心跳超过 200ms 时，自动将发送间隔从 30ms 放宽至 50ms 或 100ms。

后端网关在将操作转发至 Redis Pub/Sub 时，同样实施细粒度的批处理策略：在固定时间窗口（如 10ms）内累积来自同一房间的所有上行操作，以单条批量消息形式通过 Redis 发布。对于同一笔触的多条 `stroke_move` 消息，后端将其 `points` 数组合并为一条消息后转发，减少广播次数。Nginx 层需配置 `proxy_buffering` 与 `proxy_flush` 参数，避免大批量消息在代理层的缓冲导致的延迟叠加。

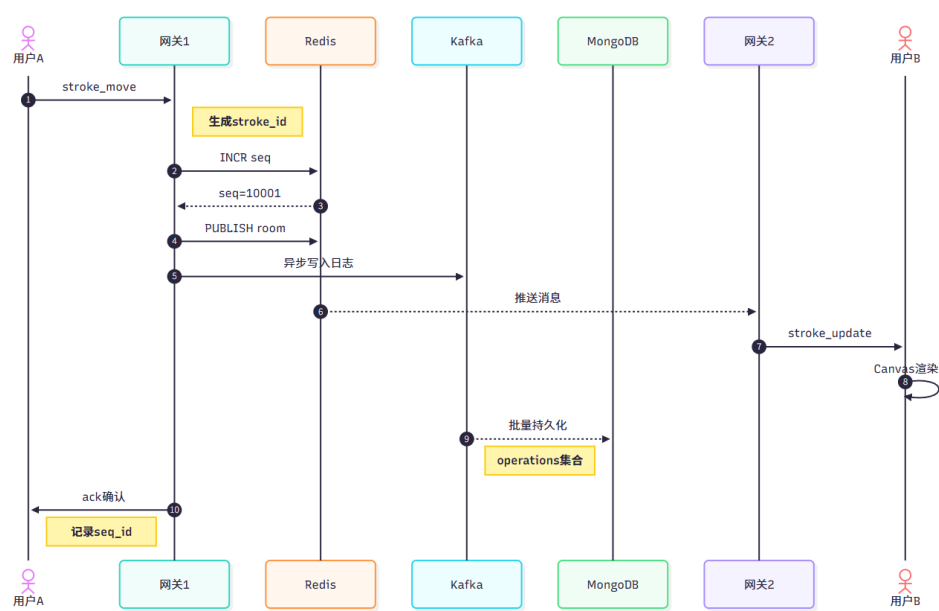


图 3.1: 跨节点笔触广播时序图

第四章 接口设计规范

4.1 HTTP RESTful 接口设计

本节描述系统对外暴露的 HTTP REST 接口，采用 JSON 格式进行请求与响应。

房间管理接口。 POST /api/v1/rooms 用于创建房间，请求体包含 name（字符串，最长 64 字符）、max_members（整数，范围 1-200）、canvas_width 和 canvas_height，响应返回 room_id（UUID）、invite_code（6 位字符串）和 access_token（JWT）。GET /api/v1/rooms/{room_id} 返回房间元数据、当前在线人数及状态（open/closed/readonly），房间信息对未认证用户也可见。POST /api/v1/rooms/{room_id}/join 在请求头携带 JWT，响应返回 WebSocket 连接地址 ws://host/ws/room/{room_id} 和初始同步信息。POST /api/v1/rooms/{room_id}/clear 仅限管理员调用，生成一条 operation_type=clear_canvas 的系统操作并广播至所有在线用户。

操作同步接口。 GET /api/v1/rooms/{room_id}/operations 以 since_sequence_id（起始序列号，默认 0）和 limit（最大条数，默认 1000，上限 5000）为查询参数，返回分页操作集，每页响应包含 operations 数组、next_since 游标和 has_more 布尔值。GET /api/v1/rooms/{room_id}/snapshot 返回最近一次画布快照（SVG 格式）和对应的 sequence_id，用于新用户快速入房。

成员与导出接口。 GET /api/v1/rooms/{room_id}/members 返回所有在线成员的用户_id、nickname、role、当前光标位置（User Presence）和最后活跃时间。GET /api/v1/rooms/{room_id}/export 以 format(svg/png)、start_seq、end_seq 和 dpi（仅 PNG）为查询参数，返回异步任务 ID，客户端通过回调或轮询获取下载链接。

认证接口。 POST /api/v1/auth/login 请求体包含 username 和 password，响应返回 JWT 令牌（有效期 2 小时）和刷新令牌（有效期 7 天）。POST /api/v1/auth/register 请求体包含 username、password 和 nickname，响应返回用户 ID 与初始 JWT。

4.2 WebSocket 事件交互协议

WebSocket 连接建立流程如下：

1. 客户端通过 HTTP GET 请求向 /ws/room/{room_id} 发起握手请求，并在请

求头 `Authorization` 中附带 JWT 令牌。

2. 网关验证 JWT 有效性，验证通过后返回 HTTP 101 Switching Protocols。
3. 连接建立后，服务器立即推送 `sync_response` 消息，包含房间初始状态（成员列表、最近操作等）。
4. 此后进入双向实时通信阶段。

心跳包（Ping/Pong）维持长连接的机制设计如下。客户端在无任何业务消息发送的情况下，每 25 秒主动向服务器发送一条 ping 消息，以防止中间设备（如企业防火墙、NAT）因 TCP 空闲超时而关闭连接。服务器在 30 秒内未收到客户端任何消息（包括 ping 或业务消息）时，将主动关闭该 WebSocket 连接并清理相关资源。Nginx 层配置 `proxy_read_timeout` 为 300 秒，确保长连接不被代理层意外关闭，该值应显著大于客户端心跳间隔。客户端重连时采用指数退避策略，最大间隔不超过 30 秒，总重试时间不超过 5 分钟。

第五章 故障演练与压测预案

5.1 核心监控大盘指标 (Observability)

系统上线后，需建立完善的可观测性 (Observability) 监控体系，覆盖以下四个黄金信号 (Golden Signals)。

延迟指标。笔触端到端 P99 延迟的告警阈值为超过 50ms，采集方式为客户端记录每条消息的发送时间戳与渲染完成时间，通过独立遥测通道上报至 Prometheus。Redis Pub/Sub 延迟 P99 超过 5ms 时告警，通过 Redis MONITOR 或 Redis Insight 采样。Kafka 消费 lag 单消费者超过 10,000 条时告警，通过 Kafka JMX Exporter 采集。

流量指标。房间活跃连接数按房间维度聚合，峰值人数告警阈值按房间容量 90% 设定。消息吞吐量全局 QPS 低于预估峰值 50% 时触发异常下跌告警。入房速率突增 300% 时触发扩容告警。

错误率指标。WebSocket 连接错误率超过 1% 时告警。消息丢失率通过 Sequence_ID 跳号检测，超过 0.1% 时告警。MongoDB 写入失败率超过 0.01% 时告警。

饱和度指标。CPU 利用率超过 70% 持续 5 分钟时触发扩容告警。内存利用率超过 80% 时触发 Full GC 风险告警。单节点 WebSocket 连接数超过 4,500 (容量 90%) 时触发扩容。Redis 内存使用率超过 75% 时触发告警。

5.2 分布式压测场景设计

为验证系统在极端并发条件下的稳定性与性能边界，本节设计了系统化的分布式压测方案。

5.2.1 场景一：200 人同房高频作画压测

测试目标：验证系统在单房间 200 名协作者同时高频发送绘画指令时，能否满足 $P99 \leq 50ms$ 的延迟要求。

测试方法：

1. 使用分布式压测工具（如 Locust、Taurus 或自定义 Go/Java 压测客户端），在 5 台压测机上部署 200 个虚拟用户（每台 40 个）。
2. 每个虚拟用户以 10 条/秒的频率发送 stroke_move 消息，每条消息携带 10 个坐标点。
3. 全局总消息产生速率： $200 \times 10 = 2,000$ 条/秒。
4. 同时有 50 名额外用户（观战者）连接但不发送绘画消息，仅接收广播，验证下行的消息扇出（Fan-out）能力。
5. 记录从发送端发出到接收端渲染完成的端到端延迟分布（P50/P90/P99）。
6. 监控各节点的 CPU、内存、Redis 连接数、Kafka consumer lag 等指标。

验收标准：端到端 P99 $\leq 50\text{ms}$ ，CPU 利用率 $\leq 70\%$ ，消息丢失率 $\leq 0.1\%$ 。

5.2.2 场景二：跨节点断线重连压测

测试目标：验证单个 WebSocket 网关节点宕机时，所有连接用户的自动重连时长与数据追平完整性。

测试方法：

1. 在房间内建立 100 个活跃连接。
2. 通过 `kill -9` 或网络分区（iptables 模拟）强制切断 Node-1 上的 100 条连接。
3. 测量从断连到所有用户完成重连的最大时长。
4. 验证重连后 sync_request/response 追平流程的正确性：所有用户在重连后的画布状态与断连前完全一致，通过对比 Sequence_ID 覆盖率确认。

5.2.3 场景三：Kafka 故障降级压测

测试目标：验证 Kafka 不可用时，系统能否降级为 Redis-only 模式，且数据不丢失。

测试方法：

1. 正常压测运行中，通过 `kill -9` 关闭所有 Kafka Broker。

2. 观察：Redis 缓存是否正常承接所有操作（TTL=30min 保证不丢失）；网关是否无报错继续正常运行；用户是否无感知。
3. 恢复 Kafka 后，验证 Kafka Consumer Group 能否从断点续消费 Redis 中暂存的操作日志（需实现 Redis → Kafka 的补写逻辑），最终 MongoDB 数据完整性。

推荐使用 Locust (Python) 作为压测框架。测试脚本在 on_start 阶段完成用户登录与 WebSocket 连接建立，然后通过 @task 装饰器定义绘画消息发送任务，以每 0.1 秒发送一批 stroke_move 消息的节奏向服务器发送坐标点数据。运行命令为 `locust -f locustfile.py --headless -u 200 -r 20`，可实现 200 并发用户、每秒 2,000 条绘画消息的压测负载。

第六章 系统部署与运维

6.1 容器化部署架构

系统各组件推荐使用 Docker 容器化部署，通过 Docker Compose 进行本地开发编排，Kubernetes (K8s) 进行生产环境集群管理。各组件的部署规格如下：

WS Gateway（镜像 `gateway:latest`，部署不少于 3 个副本，单副本资源配额 2C4G）通过环境变量配置 `Redis_HOSTS`、`KAFKA_BROKERS`、`JWT_SECRET` 和 `NODE_ID`。

Nginx LB（镜像 `nginx:alpine`，部署 2 个副本，单副本资源配额 0.5C1G）负责 TCP 负载均衡，配置 Sticky Session (`ip_hash`) 和 WSS 代理。

Redis Cluster（镜像 `redis:7-alpine`，部署 6 个节点构成 3 主 3 从架构，单节点资源配额 1C2G）开启密码认证，配置 AOF + RDB 持久化。

Kafka（镜像 `confluentinc/cp-kafka:7.5`，部署 3 个副本，单副本资源配额 2C4G）配置 Replication Factor=3，消息保留 7 天，Topic 名称为 `room_operations`。

MongoDB（镜像 `mongo:6`，部署 3 个节点构成 1 主 2 从 Replica Set，单节点资源配额 4C8G）使用 WiredTiger 存储引擎，在 `sequence_id` 和 `room_id` 字段上建立索引。